

LEVEL 12 — Pandas GroupBy

Beginner-friendly line-by-line guide with examples and outputs.

1. What is GroupBy?

GroupBy divides data into groups, performs a calculation on each group, and combines the results.

Think: Split → Apply → Combine.

Code:

```
import pandas as pd

df = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR", "Sales", "Sales"],
    "Employee": ["A", "B", "C", "D", "E", "F"],
    "Salary": [50000, 60000, 45000, 55000, 40000, 50000]
})

print(df)
```

Output:

	Department	Employee	Salary
0	IT	A	50000
1	IT	B	60000
2	HR	C	45000
3	HR	D	55000
4	Sales	E	40000
5	Sales	F	50000

2. groupby()

groupby() creates groups based on a column.

Code:

```
result = df.groupby("Department")["Salary"].sum()
print(result)
```

Output:

```
Department
HR      100000
IT      110000
Sales    90000
Name: Salary, dtype: int64
```

3. Group by one column

Group by Department and calculate the average salary.

Code:

```
result = df.groupby("Department")["Salary"].mean()
print(result)
```

Output:

```
Department
HR      50000.0
IT      55000.0
Sales    45000.0
```

Name: Salary, dtype: float64

4. Group by multiple columns

You can group using two or more columns.

Code:

```
df2 = pd.DataFrame({
    "Department": ["IT", "IT", "HR", "HR", "IT", "HR"],
    "Gender": ["M", "F", "F", "M", "M", "F"],
    "Salary": [50000, 60000, 45000, 55000, 70000, 50000]
})

result = df2.groupby(["Department", "Gender"])["Salary"].mean()
print(result)
```

Output:

Department	Gender	
HR	F	47500.0
	M	55000.0
IT	F	60000.0
	M	60000.0

Name: Salary, dtype: float64

5. sum()

sum() adds values inside each group.

Code:

```
result = df.groupby("Department")["Salary"].sum()
print(result)
```

Output:

HR	100000
IT	110000
Sales	90000

6. mean()

mean() calculates the average of each group.

Code:

```
result = df.groupby("Department")["Salary"].mean()
print(result)
```

Output:

HR	50000.0
IT	55000.0
Sales	45000.0

7. count()

count() counts non-missing values in a column.

Code:

```
result = df.groupby("Department")["Salary"].count()
print(result)
```

Output:

HR	2
IT	2
Sales	2

8. min()

min() finds the smallest value in each group.

Code:

```
result = df.groupby("Department")["Salary"].min()
print(result)
```

Output:

HR	45000
IT	50000
Sales	40000

9. max()

max() finds the largest value in each group.

Code:

```
result = df.groupby("Department")["Salary"].max()
print(result)
```

Output:

HR	55000
IT	60000
Sales	50000

10. median()

median() finds the middle value of each group.

Code:

```
result = df.groupby("Department")["Salary"].median()
print(result)
```

Output:

HR	50000.0
IT	55000.0
Sales	45000.0

11. std()

std() calculates the standard deviation for each group.

Code:

```
result = df.groupby("Department")["Salary"].std()
print(result)
```

Output:

HR	7071.067812
IT	7071.067812
Sales	7071.067812

12. size()

size() counts the total number of rows in each group, including rows with missing values.

Code:

```
result = df.groupby("Department").size()
print(result)
```

Output:

```
HR      2
IT      2
Sales   2
```

13. value_counts()

value_counts() counts how many times each value appears.

Code:

```
result = df["Department"].value_counts()
print(result)
```

Output:

```
Department
IT      2
HR      2
Sales   2
Name: count, dtype: int64
```

14. agg()

agg() lets you apply an aggregation function to grouped data.

Code:

```
result = df.groupby("Department")["Salary"].agg("mean")
print(result)
```

Output:

```
HR      50000.0
IT      55000.0
Sales   45000.0
```

15. Multiple aggregations

Use a list when you want several calculations at the same time.

Code:

```
result = df.groupby("Department")["Salary"].agg(
    ["sum", "mean", "min", "max", "count"]
)
print(result)
```

Output:

```
      sum    mean  min  max  count
Department
HR    100000  50000.0  45000  55000      2
IT    110000  55000.0  50000  60000      2
Sales   90000  45000.0  40000  50000      2
```

16. Named aggregation

Named aggregation gives meaningful names to the result columns.

Code:

```
result = df.groupby("Department").agg(
    total_salary=("Salary", "sum"),
    average_salary=("Salary", "mean"),
    highest_salary=("Salary", "max")
)

print(result)
```

Output:

	total_salary	average_salary	highest_salary
Department			
HR	100000	50000.0	55000
IT	110000	55000.0	60000
Sales	90000	45000.0	50000

17. transform()

transform() returns a result with the same number of rows as the original DataFrame.

Example: add each employee's department average.

Code:

```
df["Dept_Avg"] = df.groupby("Department")["Salary"].transform("mean")

print(df[["Department", "Employee", "Salary", "Dept_Avg"]])
```

Output:

	Department	Employee	Salary	Dept_Avg
0	IT	A	50000	55000.0
1	IT	B	60000	55000.0
2	HR	C	45000	50000.0
3	HR	D	55000	50000.0
4	Sales	E	40000	45000.0
5	Sales	F	50000	45000.0

18. filter()

filter() keeps or removes complete groups based on a condition.

Example: keep departments whose average salary is greater than 48000.

Code:

```
result = df.groupby("Department").filter(
    lambda group: group["Salary"].mean() > 48000
)

print(result)
```

Output:

	Department	Employee	Salary
0	IT	A	50000
1	IT	B	60000
2	HR	C	45000
3	HR	D	55000

Sales is removed because its average salary is 45000.

19. apply()

apply() lets you run a custom function on each group.

Code:

```
def salary_range(group):
    return group["Salary"].max() - group["Salary"].min()

result = df.groupby("Department").apply(salary_range)
print(result)
```

Output:

```
Department
HR          10000
IT          10000
Sales       10000
dtype: int64
```

20. GroupBy with reset_index()

reset_index() converts the grouping index back into a normal DataFrame column.

Code:

```
result = df.groupby("Department")["Salary"].sum().reset_index()
print(result)
```

Output:

```
  Department  Salary
0          HR  100000
1          IT  110000
2         Sales   90000
```

21. Multiple columns + multiple aggregations

This is a common real-world pattern for data analysis.

Code:

```
result = df2.groupby("Department").agg(
    total_salary=("Salary", "sum"),
    average_salary=("Salary", "mean"),
    employee_count=("Salary", "count"),
    minimum_salary=("Salary", "min"),
    maximum_salary=("Salary", "max")
)

print(result)
```

Output:

	total_salary	average_salary	employee_count	minimum_salary	maximum_salary
Department					
HR	150000	50000.0	3	45000	55000
IT	180000	60000.0	3	50000	70000

22. count() vs size()

count() counts non-null values in the selected column.

size() counts rows in each group.

Code:

```
df3 = pd.DataFrame({
    "Department": ["IT", "IT", "HR"],
    "Salary": [50000, None, 45000]
})

print(df3.groupby("Department")["Salary"].count())
print(df3.groupby("Department").size())
```

Output:

```
count():
Department
HR      1
IT      1
```

```
size():
Department
HR      1
IT      2
```

IT has 2 rows, but only 1 non-null Salary value.

23. Quick interview summary

groupby() → creates groups

sum() → total

mean() → average

count() → non-null count

min() → smallest value

max() → largest value

median() → middle value

std() → standard deviation

size() → number of rows

value_counts() → frequency of values

agg() → multiple/custom aggregations

transform() → result stays aligned with original rows

filter() → keeps/removes complete groups

apply() → custom function on each group